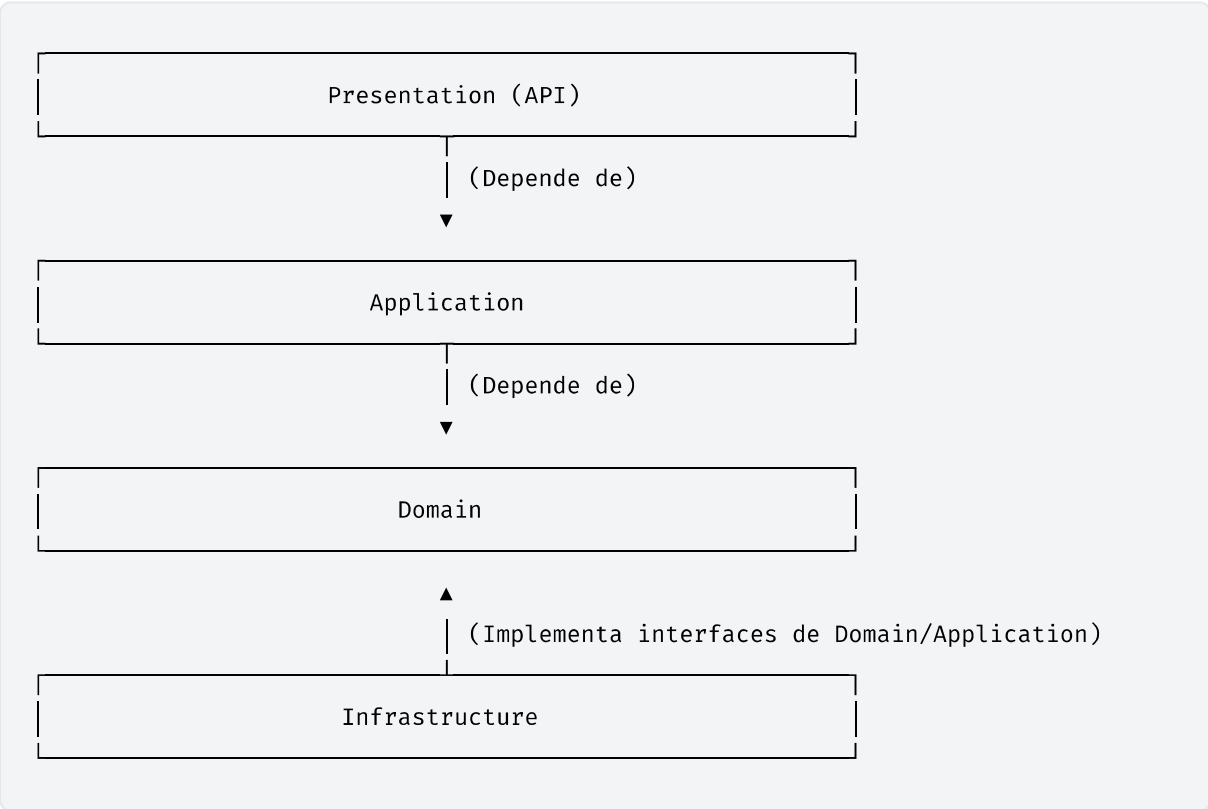


Especificación de Arquitectura Backend: .NET 8 / SQL Server 2022

Este documento establece las directrices de diseño, patrones, capas, módulos y políticas de seguridad para el backend del **Sistema Inteligente de Reclutamiento para Nacional Seguros**. Cumple en su totalidad con lo estipulado en la [Constitución del Proyecto](#).

1. Capas de la Clean Architecture (.NET 8)

El backend se dividirá en cuatro proyectos/capas principales para evitar acoplamientos y dependencias circulares:



1.1 Domain (Dominio)

- **Contenido:** Entidades de negocio, Agregados, Value Objects, Interfaces de Repositorios, Eventos de Dominio y Excepciones de Dominio.
- **Dependencias:** Ninguna (Capa pura sin referencias a frameworks, bases de datos o tecnologías externas).

1.2 Application (Aplicación)

- **Contenido:** Casos de Uso (Mediator Handlers / CQRS Command & Queries), DTOs de Entrada/Salida, Interfaces de Servicios Externos (ej. n8n, Email, SMS), Validadores (FluentValidation) y Mapeadores (AutoMapper).
- **Dependencias:** Solo depende de **Domain**.

1.3 Infrastructure (Infraestructura)

- **Contenido:** Acceso a Base de Datos (Entity Framework Core / Dapper / Repositorios), Implementación de Servicios Externos (HTTP clients para n8n, SMTP, etc.), Proveedor de JWT, Logging y Caché.
- **Dependencias:** Depende de **Domain** y **Application**.

1.4 Presentation (Presentación - API)

- **Contenido:** Controllers REST (ASP.NET Core Web API), Middlewares (Global Exception Handling, JWT validation), Swagger/OpenAPI Configuration y Dependency Injection Bootstrapper.
- **Dependencias:** Depende de **Application** e **Infrastructure** (solo para DI).

2. Estándares y Patrones de Desarrollo

- **Repository & Unit of Work (UoW):** Para encapsular la lógica de acceso a datos y asegurar que las transacciones múltiples se completen de forma atómica.
- **CQRS (Command Query Responsibility Segregation):** Mediante la librería MediatR, separando las operaciones de lectura (Queries) de las de escritura (Commands) para maximizar la mantenibilidad.
- **Validation Pipeline Behavior:** Las validaciones de entrada se ejecutan automáticamente en el pipeline de MediatR mediante FluentValidation antes de que el Command/Query llegue a su Handler.
- **Global Exception Middleware:** Captura cualquier excepción no controlada del sistema y devuelve una respuesta estructurada estándar.

3. Seguridad Backend

- **Autenticación Híbrida JWT:** El sistema admite dos flujos de autenticación controlados por el campo `TipoAutenticacion` de la tabla `Usuario` :
 - **Local** : Validación de credenciales contra el `ClaveHash` (Argon2id/BCrypt) almacenado en la base de datos. Al autenticarse, el backend emite un JWT firmado

asimétricamente (ej. RS256) con expiración corta (15-30 min) y Refresh Token almacenado en base de datos.

- **ActiveDirectory** : Validación delegada a **Active Directory** (LDAP corporativo) o **Microsoft Entra ID** (Azure AD vía OIDC). El backend valida el token externo recibido, busca al usuario por `Correo` o `ActiveDirectoryId` , obtiene sus roles de la tabla `UsuarioRol` y emite un JWT interno del sistema con los mismos claims. No se almacena contraseña local.
- **Autorización basada en Roles y Políticas (RBAC)**: Uso de atributos `[Authorize(Roles = "RRHH,Administrador")]` y políticas basadas en Claims. Los roles siempre se resuelven desde la base de datos local, independientemente del proveedor de identidad.
- **Protección de Datos**:
 - Contraseñas locales hasheadas usando **BCrypt** o **Argon2id**. Para usuarios de Active Directory, `ClaveHash` es `NULL` .
 - Cadenas de conexión, credenciales del servidor LDAP (DN de servicio, contraseña) y claves de Azure AD (`TenantId` , `ClientId` , `ClientSecret`) inyectadas exclusivamente desde **Azure Key Vault** o variables de entorno seguras. Nunca hardcodeadas.
 - La configuración de autenticación externa se define en `appsettings.json` (ambiente no productivo) y en Key Vault (ambientes QA y PROD):

```
"AzureAd": {
  "Instance": "https://login.microsoftonline.com/",
  "TenantId": "[SECRET-TENANT-ID]",
  "ClientId": "[SECRET-CLIENT-ID]"
},
"LdapSettings": {
  "Server": "dc01.nacionalseguros.local",
  "Port": 636,
  "BaseDn": "OU=Usuarios,DC=nacionalseguros,DC=local",
  "ServiceAccountDn": "[SECRET-DN]",
  "ServiceAccountPassword": "[SECRET-PASSWORD]"
}
```

4. Gestión de Errores y Formato de Respuesta Estándar

Toda API del sistema devolverá un formato de respuesta estándar en caso de fallo:

```
{
  "Code": "INVALID_ARGUMENTS",
  "Message": "Uno o más campos de la solicitud no son válidos.",
  "Detail": "El campo 'Email' es requerido y debe tener un formato válido.",
  "CorrelationId": "f7a391cb-d3f3-4d6d-b8d9-2ef53ea493b8"
}
```

- **CorrelationId:** Se genera en cada request de entrada y se propaga a los logs estructurados para facilitar el rastreo de errores en producción.

5. Auditoría Integrada

Cada escritura en base de datos (Insert, Update, Delete) deberá registrar automáticamente los cambios a través del interceptor de Entity Framework Core o el repositorio correspondiente:

Auditoría Básica (En cada entidad)

- `CreatedBy` (Usuario que realiza la acción, extraído del JWT Claim).
- `CreatedDate` (Fecha/Hora UTC del servidor).
- `ModifiedBy` (Usuario de la modificación, del JWT Claim).
- `ModifiedDate` (Fecha/Hora UTC de modificación).

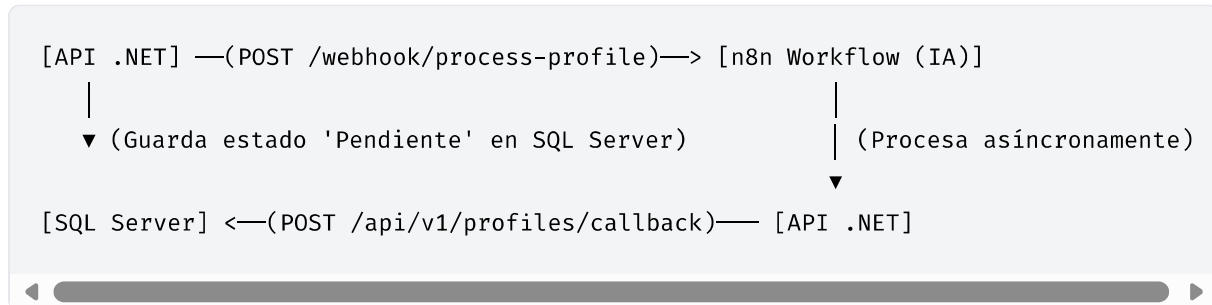
Registro de Auditoría Detallado (`AuditLog`)

Toda acción relevante escribe un registro persistente con:

- `Usuario` (Identificador o Username)
 - `Rol`
 - `Fecha`
 - `Acción` (ej. "APROBAR_SOLICITUD", "MODIFICAR_VACANTE")
 - `Módulo` (ej. "Solicitudes", "Vacantes")
 - `EstadoAnterior` (Representación serializada en JSON, si aplica)
 - `EstadoNuevo` (Representación serializada en JSON, si aplica)
 - `Canal` (ej. "Web", "WhatsApp", "Correo")
 - `Observación`
-

6. Integración con n8n (Agentes e IA)

La comunicación con n8n se realiza como un servicio externo REST, manteniendo la asincronía en tareas pesadas:



Contrato de Integración .NET ↔ n8n

- **Salida (.NET → n8n):** Headers de autorización JWT/API Key, payload JSON estructurado con el contexto requerido por la IA.
- **Entrada (n8n → .NET):** Webhook callback expuesto en .NET que recibe el resultado de la IA. El endpoint de callback debe requerir token de autenticación API Key específico del sistema, validar la firma o procedencia del request y estar protegido por el middleware de **Idempotencia de Callbacks** (`IdempotencyFilter`).
- **Idempotencia de Callbacks:** El `IdempotencyFilter` valida la cabecera `X-Correlation-ID` en las llamadas entrantes a los controladores de callback. Si la petición ya está en procesamiento o fue completada en los últimos 60 segundos, el middleware intercepta la llamada y responde con éxito inmediato (`200 OK` o `202 Accepted`), evitando reprocesamientos e inserciones dobles en `StateHistory` .
- **Registro de Ejecución de Agentes IA (`AgentExecution`):** Registra el resultado del webhook de callback, duración y estado final.

7. Módulos Core a Desarrollar

1. **Solicitudes:** Flujos de aprobación, estados (`Borrador` , `PendienteAprobacion` , `Aprobado` , `Rechazado`).
2. **Perfiles:** Creación, versionado de profesigramas, validación por IA y aprobación del decisor.
3. **Vacantes:** Apertura, publicación, monitoreo de SLAs (tiempo para cubrir la vacante).
4. **Postulantes:** Carga de CV, parsing de CV (vía n8n/IA), scoring de adecuación, listado y filtrado.
5. **Agenda:** Coordinación de entrevistas, envío automático de recordatorios por WhatsApp y Correo (enviados por n8n pero gatillados por APIs .NET).
6. **Parametrización:** Catálogos de roles, estados, plantillas y prompts de IA versionados.

7. **Auditoría:** Vista de logs de auditoría e historial del sistema para el rol Auditor.